МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение
высшего образования
«МИРЭА – Российский технологический университет»

РТУ МИРЭА

Институт информационных технологий (ИТ)
Кафедра инструментального и прикладного программного обеспечения (ИиППО)

ЗАДАНИЕ
на выполнение курсовой работы

по дисциплине: Бэкенд-разработка
по профилю: Разработка программных продуктов и проектирование информационных систем
направления профессиональной подготовки: Программная инженерия (09.03.04)

Студент: Данов Арсений Иванович
Группа: ИКБО-10-23
Срок представления к защите: 13.05.2026
Руководитель: Волков Михаил Юрьевич, ст. преподаватель

Тема: Веб-сервис по управлению персоналом

Исходные данные: используемые технологии: языки программирования: python, фреймворки: Django, среды разработки: Visual Studio Code, SQL СУБД PostgreSQL, наличие: межстраничной навигации, внешнего вида страниц, соответствующего современным стандартам веб-разработки, использование паттерна проектирования (MVT). Нормативный документ: инструкция по организации и проведению курсового проектирования СМКО МИРЭА 7.5.1/04.И.05-18.

Перечень вопросов, подлежащих разработке, и обязательного графического материала: 1. Провести анализ предметной области разрабатываемого веб-приложения. 2. Обосновать выбор технологий разработки веб-приложения. 3. Разработать архитектуру веб-приложения на основе выбранного паттерна проектирования. 4. Реализовать слой серверной логики веб-приложения с применением выбранной технологии. 5. Реализовать слой логики базы данных. 6. Разработать слой клиентского представления веб-приложения. 7. Создать презентацию по выполненной курсовой работе. 8. Оформить отчет в соответствии с ГОСТ 7.32-2017.

Руководителем произведён инструктаж по технике безопасности, противопожарной технике и правилам внутреннего распорядка.

Зав. кафедрой ИиППО: _____/Р. Г. Болбаков/, «20» февраля 2026 г.

Задание на КР выдал: _____/М.Ю. Волков/, «20» февраля 2026 г.

Задание на КР получил: _____/А.И. Данов/, «20» февраля 2026 г.

Заведующему кафедрой

инструментального и прикладного

программного обеспечения (ИиППО)

Института информационных технологий (ИТ)

Болбакову Роману Геннадьевичу

От студента

Данова Арсения Ивановича

группы ИКБО-10-23

3 курса

Контакт: danov2005@yandex.ru

Заявление

Прошу утвердить мне тему *курсовой работы* по дисциплине «*Бэкенд-разработка*» образовательной программы бакалавриата *09.03.04 (Программная инженерия)*

Тема: Веб-сервис по управлению персоналом

Приложение: лист задания на КР в 2-ух экземплярах на двухстороннем листе (проект)

Подпись студента		Данов А.И.
	подпись	*ФИО*
Дата	20.02.2026г.	

Подпись руководителя		ст. преп. Волков М.Ю.
	подпись	*Должность, ФИО*
Дата	20.02.2026г.	

Согласовано Зав. кафедрой ИиППО 20.02.2026г._____ / Болбаков Р.Г/

МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение высшего образования

«МИРЭА – Российский технологический университет»

РТУ МИРЭА

Институт информационных технологий (ИТ)

Кафедра инструментального и прикладного программного обеспечения (ИиППО)

ЗАДАНИЕ
на выполнение курсовой работы

по дисциплине: Бэкенд-разработка
по профилю: Разработка программных продуктов и проектирование информационных систем
направления профессиональной подготовки: Программная инженерия (09.03.04)

Студент: Данов Арсений Иванович
Группа: ИКБО-10-23
Срок представления к защите: 13.05.2026
Руководитель: Волков Михаил Юрьевич, ст. преподаватель

Тема: Веб-сервис по управлению персоналом

Исходные данные: используемые технологии: языки программирования: python, фреймворки: Django, среды разработки: Visual Studio Code, SQL СУБД PostgreSQL, наличие: межстраничной навигации, внешнего вида страниц, соответствующего современным стандартам веб-разработки, использование паттерна проектирования (MVT). Нормативный документ: инструкция по организации и проведению курсового проектирования СМКО МИРЭА 7.5.1/04.И.05-18.

Перечень вопросов, подлежащих разработке, и обязательного графического материала: 1. Провести анализ предметной области разрабатываемого веб-приложения. 2. Обосновать выбор технологий разработки веб-приложения. 3. Разработать архитектуру веб-приложения на основе выбранного паттерна проектирования. 4. Реализовать слой серверной логики веб-приложения с применением выбранной технологии. 5. Реализовать слой логики базы данных. 6. Разработать слой клиентского представления веб-приложения. 7. Создать презентацию по выполненной курсовой работе. 8. Оформить отчет в соответствии с ГОСТ 7.32-2017.

Руководителем произведён инструктаж по технике безопасности, противопожарной технике и правилам внутреннего распорядка.

Зав. кафедрой ИиППО: _____/Р. Г. Болбаков/, «20» февраля 2026 г.

Задание на КР выдал: _____/М.Ю. Волков/, «20» февраля 2026 г.

Задание на КР получил: _____/А.И. Данов/, «20» февраля 2026 г.

РЕФЕРАТ

Отчёт 36 с., 8 рис., 3 табл., 18 ист.

ВЕБ-СЕРВИС, УПРАВЛЕНИЕ ПЕРСОНАЛОМ, БЭКЕНД-РАЗРАБОТКА, DJANGO, MVT

Данов А.И., курсовая работа по направлению подготовки «Программная инженерия» по дисциплине «Бэкенд-разработка» на тему «Веб-сервис по управлению персоналом»: М. 2026 г., МИРЭА - Российский технологический университет (РТУ МИРЭА), Институт информационных технологий (ИИТ), кафедра инструментального и прикладного программного обеспечения (ИиППО).

Целью работы является разработка серверной части веб-сервиса по управлению персоналом. В работе проведен анализ предметной области, сформированы функциональные и нефункциональные требования, обоснован выбор технологий разработки, разработана архитектура приложения на основе паттерна MVT, реализована серверная часть веб-сервиса на фреймворке Django. Разработанный сервис обеспечивает учет сотрудников, регистрацию выполненных задач, фиксацию затраченного времени, хранение отчетов и просмотр статистики по сотрудникам для руководителя.

СОДЕРЖАНИЕ

ОПРЕДЕЛЕНИЯ, ОБОЗНАЧЕНИЯ И СОКРАЩЕНИЯ 7
1 ВВЕДЕНИЕ 9
2 АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ 11
2.1 Характеристика предметной области 11
2.2 Основные участники и бизнес-процессы 11
2.3 Анализ существующих решений 12
2.4 Формирование требований к системе 14
2.4.1 Функциональные требования 14
2.4.2 Нефункциональные требования 15
2.4.3 Матрица соответствия требований 15
2.5 Выводы по главе 16
3 ВЫБОР И ОБОСНОВАНИЕ ТЕХНОЛОГИЙ 17
3.1 Выбор серверного фреймворка 17
3.2 Выбор системы управления базами данных 18
3.3 Выбор средства кеширования 19
3.4 Выбор средства развертывания 19
3.5 Выбор архитектурного паттерна 20
3.6 Выводы по главе 21
4 РАЗРАБОТКА АРХИТЕКТУРЫ ПРИЛОЖЕНИЯ НА ОСНОВЕ ВЫБРАННОГО ПАТТЕРНА 22
4.1 Обоснование применения паттерна MVT 22
4.2 Общая архитектура веб-сервиса 22
4.3 Основные сущности и их связи 23
4.4 Компоненты приложения в терминах MVT 24
4.5 Пользовательские сценарии работы системы 25
4.6 Выводы по главе 26
5 РАЗРАБОТКА СЕРВЕРНОЙ ЧАСТИ ИНТЕРНЕТ-РЕСУРСА 27
5.1 Реализация моделей данных 27
5.2 Реализация форм и валидации 27
5.3 Реализация представлений и маршрутизации 28
5.4 Реализация аутентификации и разграничения доступа 29

5.5 Реализация статистики и расчетов 30
5.6 Кеширование и запуск приложения 30
5.7 Демонстрация работы веб-сервиса 31
5.8 Выводы по главе 32
Заключение 34
Список использованных источников 35

ОПРЕДЕЛЕНИЯ, ОБОЗНАЧЕНИЯ И СОКРАЩЕНИЯ

В настоящем отчете применяют следующие определения, обозначения и сокращения.

API (Application Programming Interface)	– программный интерфейс приложения, обеспечивающий взаимодействие между программными компонентами.
CRUD (Create, Read, Update, Delete)	– базовые операции создания, чтения, изменения и удаления данных.
CSS (Cascading Style Sheets)	– язык описания внешнего вида веб-страниц.
Django	– высокоуровневый веб-фреймворк на языке Python, предназначенный для разработки серверной части веб-приложений.
Docker	– программная платформа контейнеризации, используемая для упаковки и запуска приложений в изолированной среде.
HTML (HyperText Markup Language)	– язык разметки веб-страниц.
HTTP (HyperText Transfer Protocol)	– протокол передачи данных между клиентом и сервером.
MVT (Model-View-Template)	– архитектурный паттерн, используемый в Django и разделяющий приложение на модели данных, обработчики запросов и шаблоны отображения.
ORM (Object-Relational Mapping)	– технология отображения объектов языка программирования на таблицы реляционной базы данных.
PostgreSQL	– объектно-реляционная система управления базами данных.
Redis	– высокопроизводительное хранилище данных в памяти, применяемое в том числе для кеширования.
SQL (Structured Query Language)	– язык запросов к реляционным базам данных.

URL (Uniform Resource Locator)	– унифицированный указатель адреса ресурса в сети.
СУБД	– система управления базами данных.
Фреймворк	– программный каркас, задающий базовую структуру и инструменты разработки программного обеспечения.

1 ВВЕДЕНИЕ

В условиях цифровизации деятельности организаций задачи учета трудозатрат, контроля выполнения работ и формирования прозрачной отчетности по сотрудникам приобретают особую значимость. Ручное ведение таких данных в таблицах или разрозненных документах повышает вероятность ошибок, затрудняет контроль сроков и усложняет получение актуальной управленческой информации. По этой причине востребованными становятся веб-сервисы, позволяющие централизованно хранить сведения о сотрудниках, выполненных задачах, затраченном времени и результатах работы [1].

Тема управления персоналом в целом значительно шире, чем задачи, рассматриваемые в настоящей курсовой работе. Полноценные HR-системы могут включать кадровый документооборот, подбор персонала, управление отпусками, оценку компетенций, обучение и иные процессы. В рамках данной работы предметная область сужена до задач учета выполненных работ, фиксации затраченного рабочего времени, формирования сводной статистики по сотрудникам и контроля результативности их деятельности.

Актуальность темы обусловлена необходимостью разработки серверной части интернет-ресурса, обеспечивающей надежное хранение данных, обработку пользовательских запросов, разграничение прав доступа и формирование сводной статистики. Для решения подобных задач в учебной и практической разработке широко применяется фреймворк Django, предоставляющий средства для реализации моделей данных, обработки HTTP-запросов, аутентификации и работы с шаблонами представления информации [2].

Целью курсовой работы является разработка серверной части веб-сервиса по управлению персоналом.

Для достижения поставленной цели необходимо решить следующие задачи:

– провести анализ предметной области разрабатываемого веб-приложения;

– сформировать функциональные и нефункциональные требования к разрабатываемому веб-сервису;

– обосновать выбор технологий разработки веб-приложения;

– разработать архитектуру веб-приложения на основе паттерна проектирования MVT;

– реализовать серверную часть веб-сервиса на фреймворке Django;

– описать основные пользовательские сценарии и результаты работы приложения.

Объект исследования - процессы управления персоналом в части учета задач, рабочего времени и формирования сводной информации по сотрудникам.

Предмет исследования - методы и средства разработки серверной части веб-сервиса по управлению персоналом на базе фреймворка Django.

Результатом работы является серверная часть веб-сервиса, позволяющего регистрировать сотрудников, фиксировать выполняемые ими задачи и затраченное время, хранить сопроводительные отчеты, а также предоставлять руководителю сводную статистику и информацию о начисляемых выплатах.

2 АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ

2.1 Характеристика предметной области

Управление персоналом в современных организациях связано не только с хранением сведений о сотрудниках, но и с постоянным контролем их фактической занятости, выполняемых задач и результатов работы. Особенно это важно для команд, в которых учет трудозатрат влияет на оценку эффективности сотрудника, на распределение нагрузки и на расчет вознаграждения. В таких условиях руководителю требуется получать актуальную и структурированную информацию о том, какие задачи были выполнены, сколько времени было затрачено на их выполнение и каков итоговый объем работ за определенный период [1].

Во многих небольших организациях и проектных командах учет подобных данных ведется вручную: с использованием электронных таблиц, текстовых отчетов или сообщений в корпоративных мессенджерах. Подобный подход не обеспечивает целостности данных, создает риск дублирования информации, усложняет контроль доступа и требует дополнительных временных затрат на подготовку сводной отчетности. Кроме того, ручной учет не позволяет в удобной форме связывать сведения о сотруднике, категории выполненной задачи, потраченном времени и расчетной стоимости работ [3].

Следовательно, возникает необходимость в разработке веб-сервиса, серверная часть которого будет централизованно хранить данные о сотрудниках и задачах, обеспечивать регистрацию и аутентификацию пользователей, предоставлять руководителю агрегированные сведения по каждому сотруднику и формировать информационную основу для расчета выплат. Такой сервис должен быть доступен через браузер, поддерживать разделение ролей и обеспечивать обработку типовых операций в рамках единой информационной системы.

2.2 Основные участники и бизнес-процессы

В рамках рассматриваемой предметной области выделяются следующие основные участники системы:

– сотрудник;
– руководитель;

– администратор системы.

Сотрудник взаимодействует с системой для ведения собственной трудовой отчетности. После прохождения регистрации и аутентификации он получает доступ к личному профилю, где может просматривать сводные показатели своей активности, а также добавлять сведения о выполненной задаче. Для каждой задачи фиксируются ее категория, количество затраченных часов, поясняющий комментарий и при необходимости прикрепляемый отчетный файл.

Руководитель использует систему для получения управленческой информации. В отличие от сотрудника, он должен иметь доступ не только к собственным данным, но и к сводной статистике по всем сотрудникам. На основе сохраненных данных руководитель может видеть количество выполненных задач, суммарно отработанное время и ориентировочную сумму выплат по каждому сотруднику. Также ему доступен переход к детализации задач конкретного работника.

Администратор системы обеспечивает поддержку данных и структуры приложения. В учебной реализации его функции могут выполняться через административную панель Django, что позволяет управлять учетными записями сотрудников, категориями задач и записями о выполненных работах.

Ключевые бизнес-процессы рассматриваемой системы можно представить следующим образом:

– регистрация нового сотрудника в системе;
– аутентификация сотрудника или руководителя;
– внесение сотрудником записи о выполненной задаче;
– хранение и обработка сведений о трудозатратах;
– получение руководителем сводной статистики по сотрудникам;
– просмотр детализированной информации по задачам конкретного сотрудника.

2.3 Анализ существующих решений

Для определения требований к разрабатываемому веб-сервису необходимо рассмотреть существующие программные продукты, применяемые для учета задач, рабочего времени и управленческой

отчетности. В качестве аналогов целесообразно рассмотреть решения, которые принадлежат к смежным классам систем: корпоративные платформы управления задачами, сервисы учета рабочего времени и системы внутреннего управления деятельностью сотрудников.

В качестве аналогов были выбраны следующие решения:

– Jira с модулем учета трудозатрат Tempo Timesheets;
– Bitrix24;
– Clockify.

Jira и Tempo Timesheets широко применяются в командах разработки и проектных организациях. Их сильной стороной является развитый механизм работы с задачами, проектами и отчетностью по времени. Однако такое решение ориентировано прежде всего на более сложные процессы управления проектами и может оказаться избыточным для небольших коллективов, которым требуется специализированный сервис с ограниченным, но понятным набором функций [4].

Bitrix24 представляет собой комплексную корпоративную платформу, включающую средства коммуникации, постановки задач, CRM и элементы кадрового управления. Преимуществом решения является многофункциональность, однако для локального сценария учета задач и рабочего времени сотрудников такой продукт может быть перегружен избыточными модулями, что усложняет внедрение и адаптацию [5].

Clockify ориентирован на учет рабочего времени и формирование отчетности по трудозатратам. Решение удобно для фиксации времени, но слабо покрывает задачи, связанные с хранением прикрепляемых отчетов, гибкой настройкой ролей под конкретную организацию и адаптацией структуры данных к внутренним бизнес-правилам компании [6].

Сравнительный анализ аналогов представлен в таблице 1.

Таблица 1 – Сравнительный анализ аналогов

Решение	Учет задач	Учет времени	Сводная статистика	Применимость для специализированного сервиса
Jira + Tempo Timesheets	Высокий уровень	Высокий уровень	Высокий уровень	Средняя
Bitrix24	Высокий уровень	Средний уровень	Высокий уровень	Низкая
Clockify	Низкий уровень	Высокий уровень	Средний уровень	Средняя
Разрабатываемый веб-сервис	Необходимый уровень	Необходимый уровень	Необходимый уровень	Высокая

На основании анализа можно сделать вывод, что существующие решения либо ориентированы на широкий спектр корпоративных задач, либо покрывают только часть функциональности, необходимой в рамках рассматриваемой предметной области. Разрабатываемый веб-сервис должен занять нишу специализированного решения, обеспечивающего учет сотрудников, задач, затраченного времени и расчетной стоимости выполненных работ в рамках единой серверной системы.

2.4 Формирование требований к системе

На основе анализа предметной области и рассмотренных аналогов можно сформулировать требования к разрабатываемому веб-сервису.

2.4.1 Функциональные требования

Разрабатываемый веб-сервис должен обеспечивать выполнение следующих функций:

– регистрацию пользователя в системе;

– аутентификацию сотрудника и руководителя;

– хранение учетных данных пользователя и дополнительной персональной информации;

– создание сотрудником записи о выполненной задаче;

– указание категории задачи и затраченного времени;

– добавление текстового комментария к задаче;

– прикрепление отчетного файла к задаче;

– просмотр сотрудником собственной статистики;

– просмотр руководителем списка сотрудников со сводной информацией;

– просмотр руководителем перечня задач конкретного сотрудника;

– расчет суммарного количества часов и суммы выплат на основании сохраненных записей.

2.4.2 Нефункциональные требования

К основным нефункциональным требованиям относятся:

– доступность веб-сервиса на уровне не ниже 99% в рабочее время организации;

– время формирования страницы личного профиля не более 2 с при объеме данных до 10^4 записей о задачах;

– время формирования сводной страницы статистики не более 3 с при количестве сотрудников до 100 и общем объеме данных до 10^4 записей;

– поддержка не менее 50 одновременно зарегистрированных пользователей без нарушения корректности хранения данных;

– размер одного прикрепляемого отчетного файла не более 10 МБ;

– сохранность данных при перезапуске серверной части приложения за счет использования постоянного хранилища базы данных.

2.4.3 Матрица соответствия требований

Результаты анализа требований целесообразно представить в виде матрицы соответствия, показанной в таблице 2.

Таблица 2 – Матрица соответствия требований

Требование	Сотрудник	Руководитель	Серверная часть
Регистрация и аутентификация	Да	Да	Да
Добавление информации о задаче	Да	Нет	Да
Просмотр личной статистики	Да	Нет	Да
Просмотр общей статистики	Нет	Да	Да
Просмотр задач конкретного сотрудника	Нет	Да	Да
Расчет суммарных выплат	Нет	Да	Да
Хранение отчетных файлов	Да	Да	Да

Таким образом, основная нагрузка по реализации бизнес-логики, хранению данных и разграничению доступа ложится на серверную часть

веб-приложения. Именно она должна обеспечивать целостность работы всех пользовательских сценариев.

2.5 Выводы по главе

В результате анализа предметной области установлено, что для учета задач сотрудников, рабочего времени и формирования сводной информации руководителю требуется специализированный веб-сервис с централизованным хранением данных и разграничением доступа. Рассмотрение существующих решений показало, что универсальные корпоративные платформы обладают избыточной функциональностью, а специализированные сервисы учета времени не всегда позволяют адаптировать систему под конкретные требования организации.

На основе проведенного анализа были выделены основные участники системы, определены ключевые бизнес-процессы, а также сформированы функциональные и нефункциональные требования к разрабатываемому веб-сервису. Полученные результаты служат основанием для выбора технологий реализации и проектирования архитектуры серверной части приложения.

3 ВЫБОР И ОБОСНОВАНИЕ ТЕХНОЛОГИЙ

3.1 Выбор серверного фреймворка

Разработка веб-сервиса по управлению персоналом требует выбора такого серверного фреймворка, который позволил бы в рамках одной технологии реализовать модели данных, обработку пользовательских запросов, механизмы аутентификации и авторизации, а также средства взаимодействия с реляционной базой данных. Для решения этой задачи рассмотрены наиболее распространенные Python-фреймворки: Django, Flask и FastAPI.

Flask представляет собой легковесный фреймворк, предоставляющий разработчику высокую степень свободы в выборе компонентов приложения. Его преимуществом является минимализм и простота расширения. Однако при разработке полноценного веб-сервиса с регистрацией пользователей, разграничением прав доступа, административным интерфейсом и моделью данных на основе ORM разработчику требуется самостоятельно подбирать и интегрировать значительное количество дополнительных библиотек. Это увеличивает объем конфигурации и усложняет сопровождение проекта [7].

FastAPI ориентирован прежде всего на разработку API и отличается высокой производительностью, встроенной валидацией данных и удобством документирования интерфейсов. Тем не менее в рамках рассматриваемой курсовой работы требуется построить серверную часть именно веб-приложения на Django, использующего шаблоны отображения, формы, встроенную систему аутентификации и классические средства обработки запросов. Для такого сценария FastAPI не является наиболее естественным выбором [8].

Django представляет собой высокоуровневый веб-фреймворк, который уже содержит встроенные механизмы для решения ключевых задач серверной разработки: ORM, маршрутизацию, формы, аутентификацию, административную панель и систему шаблонов [2]. Такой подход позволяет сосредоточиться непосредственно на бизнес-логике приложения и обеспечивает более быстрое создание работоспособного сервиса по сравнению с более низкоуровневыми альтернативами.

Для наглядного сравнения выбранных технологий составлена таблица 3.

Таблица 3 – Сравнение серверных фреймворков

Критерий	Django	Flask	FastAPI
Встроенная ORM	Да	Нет	Нет
Шаблоны и формы	Да	Частично	Ограниченно
Встроенная аутентификация	Да	Нет	Нет
Административная панель	Да	Нет	Нет
Степень применимости для учебного веб-сервиса	Высокая	Средняя	Средняя

По результатам сравнения для разработки серверной части веб-сервиса по управлению персоналом выбран фреймворк Django. Его использование позволяет в короткие сроки реализовать обработку пользовательских сценариев, работу с базой данных и разграничение доступа без привлечения большого числа дополнительных компонентов.

3.2 Выбор системы управления базами данных

Разрабатываемый веб-сервис должен хранить сведения о пользователях, категориях задач и самих задачах сотрудников. Для этого необходима реляционная СУБД, обеспечивающая надежное хранение данных, поддержку связей между сущностями и корректную работу агрегатных запросов. В качестве основных альтернатив целесообразно рассмотреть PostgreSQL и MySQL.

MySQL является популярной системой управления базами данных и часто применяется в веб-разработке. Она отличается простотой развертывания и хорошей производительностью в типовых сценариях. Однако PostgreSQL предоставляет более широкие возможности работы со сложными запросами, более строгий подход к обеспечению целостности данных и развитую поддержку аналитических операций [9].

Для рассматриваемого веб-сервиса важна корректная обработка связанных сущностей, выполнение агрегатных вычислений по трудозатратам и расчетным выплатам, а также надежное хранение учетных данных

пользователей. По этой причине в качестве основной СУБД выбрана PostgreSQL.

3.3 Выбор средства кеширования

При работе веб-сервиса некоторые страницы формируются на основе повторяющихся запросов к базе данных. Это особенно актуально для страниц статистики и списков задач, которые могут открываться неоднократно без мгновенного изменения содержимого. Для снижения нагрузки на серверную часть и ускорения формирования ответа в приложении используется механизм кеширования.

В качестве средства кеширования выбран Redis. Данное решение хранит данные в оперативной памяти и обеспечивает высокую скорость доступа, что делает его подходящим для кеширования результатов типовых запросов и страниц веб-приложения. Redis хорошо интегрируется с Django и позволяет быстро внедрить механизм временного хранения результатов обработки запросов [10].

3.4 Выбор средства развертывания

Для практического запуска приложения требуется удобный способ сборки и запуска его компонентов. В качестве такого средства использована контейнеризация с применением Docker. Данный инструмент позволяет запускать приложение и сопутствующие сервисы в предсказуемом окружении, снижая влияние различий между рабочими машинами и упрощая воспроизводимость результата [11].

Для обслуживания внешних запросов и публикации статических файлов в составе приложения используются Traefik и Nginx. Traefik выбран в качестве входной точки и балансировщика нагрузки, так как он удобно работает в контейнерной среде и поддерживает маршрутизацию трафика между сервисами [12]. Nginx используется как веб-сервер для отдачи статических и медиафайлов, а также для проксирования запросов к серверной части Django; такой выбор обусловлен его распространенностью и высокой эффективностью при обработке большого числа соединений [13].

Краткая схема развернутой архитектуры приложения представлена на рисунке 1.

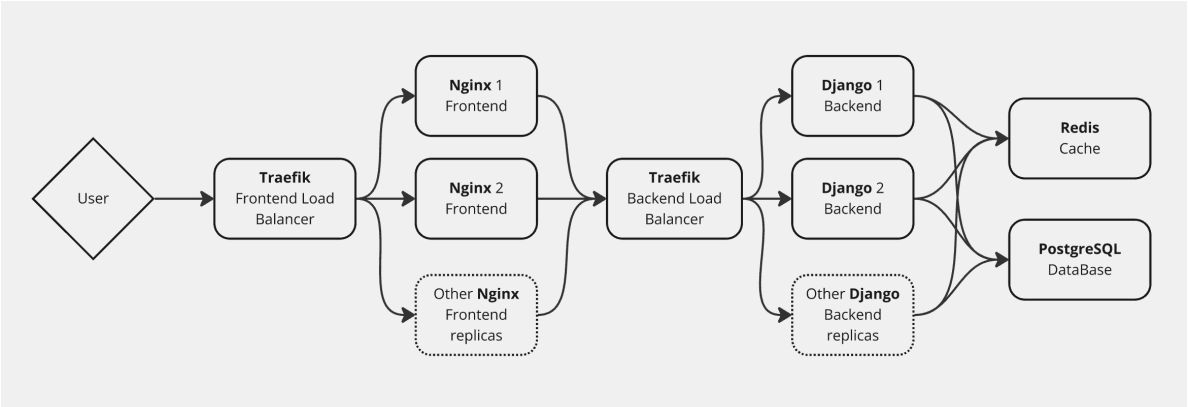

Рисунок 1 – Технологическая схема развертывания компонентов веб-сервиса

3.5 Выбор архитектурного паттерна

Поскольку разработка ведется на Django, наиболее естественным и обоснованным является применение паттерна MVT (Model-View-Template) [14].

Паттерн MVT предполагает разделение приложения на три взаимосвязанные части:

– модель (Model), отвечающую за структуру данных и взаимодействие с базой данных;

– представление (View), обрабатывающее запрос пользователя и реализующее прикладную логику;

– шаблон (Template), отвечающий за отображение подготовленных данных в интерфейсе.

Преимущество данного подхода состоит в разделении ответственности между компонентами приложения. Это упрощает сопровождение кода, делает структуру проекта более понятной и позволяет независимо изменять данные, обработку запросов и внешний вид страниц [15]. Для рассматриваемого веб-сервиса такой подход особенно удобен, поскольку в нем присутствуют явно выраженные сущности предметной области, набор отдельных пользовательских сценариев и несколько шаблонов отображения статистики и профиля пользователя.

В рамках Django паттерн MVT поддерживается самим фреймворком на уровне базовой структуры проекта. Это означает, что разработчик не внедряет его искусственно, а использует естественную архитектурную модель платформы. Следовательно, выбор MVT полностью соответствует как

используемой технологии, так и поставленной задаче по разработке серверной части веб-сервиса.

3.6 Выводы по главе

В результате анализа технологий для разработки серверной части веб-сервиса по управлению персоналом выбран фреймворк Django, обеспечивающий встроенные средства для работы с моделями, формами, шаблонами, аутентификацией и административным интерфейсом. В качестве основной системы управления базами данных выбрана PostgreSQL, позволяющая надежно хранить структурированные данные и выполнять агрегатные вычисления. Для кеширования повторяющихся запросов выбран Redis, а для воспроизводимого запуска приложения использован Docker.

В качестве архитектурного паттерна выбран MVT, так как он является базовым паттерном Django и обеспечивает логичное разделение приложения на модели данных, обработчики запросов и шаблоны отображения. Полученные результаты создают основу для проектирования архитектуры веб-сервиса и дальнейшего описания реализации его серверной части.

4 РАЗРАБОТКА АРХИТЕКТУРЫ ПРИЛОЖЕНИЯ НА ОСНОВЕ ВЫБРАННОГО ПАТТЕРНА

4.1 Обоснование применения паттерна MVT

В предыдущей главе был обоснован выбор фреймворка Django и архитектурного паттерна MVT. Применение данного паттерна в рамках разрабатываемого веб-сервиса по управлению персоналом является оправданным, поскольку функциональность системы естественным образом разделяется на три группы ответственности: хранение и описание предметных данных, обработку пользовательских запросов и отображение результатов работы в веб-интерфейсе [2].

Компонент Model в рассматриваемом проекте отвечает за структуру предметной области. В нем описываются сущности пользователя, категории задачи и записи о выполненной задаче. Именно модели обеспечивают хранение данных и связь серверной части приложения с реляционной базой данных.

Компонент View отвечает за обработку пользовательских сценариев. На этом уровне принимаются HTTP-запросы, выбирается необходимая логика, подготавливаются данные для отображения, выполняется проверка прав доступа и формируется ответ пользователю.

Компонент Template обеспечивает отображение данных в виде HTML-страниц. Благодаря шаблонам серверная часть может формировать интерфейс страницы профиля сотрудника, страницы общей статистики и страницы просмотра задач конкретного сотрудника.

Таким образом, паттерн MVT позволяет структурировать проект в соответствии с природой решаемой задачи и согласуется со стандартной архитектурой Django-приложения.

4.2 Общая архитектура веб-сервиса

Разрабатываемый веб-сервис является серверным веб-приложением, работающим по клиент-серверной модели. Пользователь взаимодействует с системой через браузер, отправляя HTTP-запросы. Серверная часть приложения принимает запрос, передает его соответствующему обработчику, взаимодействует с базой данных, а затем формирует HTML-ответ с использованием шаблона.

Общую архитектуру веб-сервиса можно представить следующей цепочкой:

– пользовательский браузер формирует HTTP-запрос;

– система маршрутизации Django определяет требуемый обработчик;

– представление обрабатывает запрос и при необходимости обращается к моделям;

– ORM выполняет чтение или изменение данных в PostgreSQL;

– представление передает подготовленные данные в шаблон;

– шаблон формирует HTML-страницу, возвращаемую пользователю.

Данная схема соответствует паттерну MVT и обеспечивает четкое разделение между данными, прикладной логикой и отображением результата.

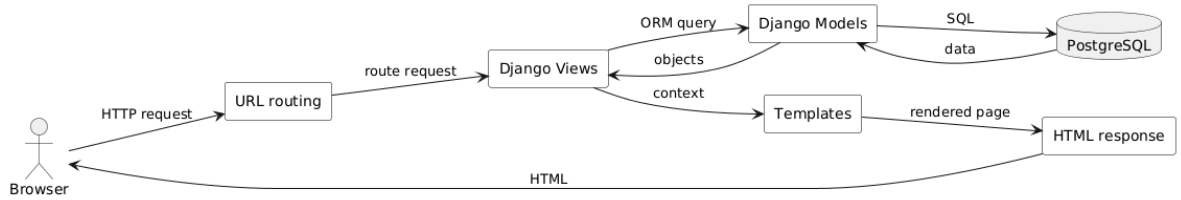

Рисунок 2 – Структурная схема взаимодействия компонентов веб-сервиса

4.3 Основные сущности и их связи

Архитектура разрабатываемого веб-сервиса строится вокруг трех ключевых сущностей предметной области:

– пользователь системы;

– категория задачи;

– задача сотрудника.

Пользователь является центральной сущностью системы. В проекте используется собственная модель пользователя, наследующая стандартную модель Django `AbstractUser`. Это позволяет сохранить встроенные механизмы аутентификации и одновременно расширить структуру данных дополнительным полем для хранения фотографии пользователя [14].

Категория задачи предназначена для описания типа выполняемой работы. Для каждой категории задаются название и стоимость, что позволяет в дальнейшем использовать эти данные при расчете суммарной выплаты сотруднику.

Сущность задачи связывает конкретного сотрудника с выполненной работой. Для нее хранятся категория, затраченное время, комментарий, дата создания и прикрепленный отчетный файл. Связь между задачей и пользователем реализована как отношение многие-к-одному: один сотрудник может иметь множество задач. Аналогично одна категория может использоваться во множестве задач.

Структура сущностей и связей может быть представлена в виде ER-диаграммы на рисунке 3.

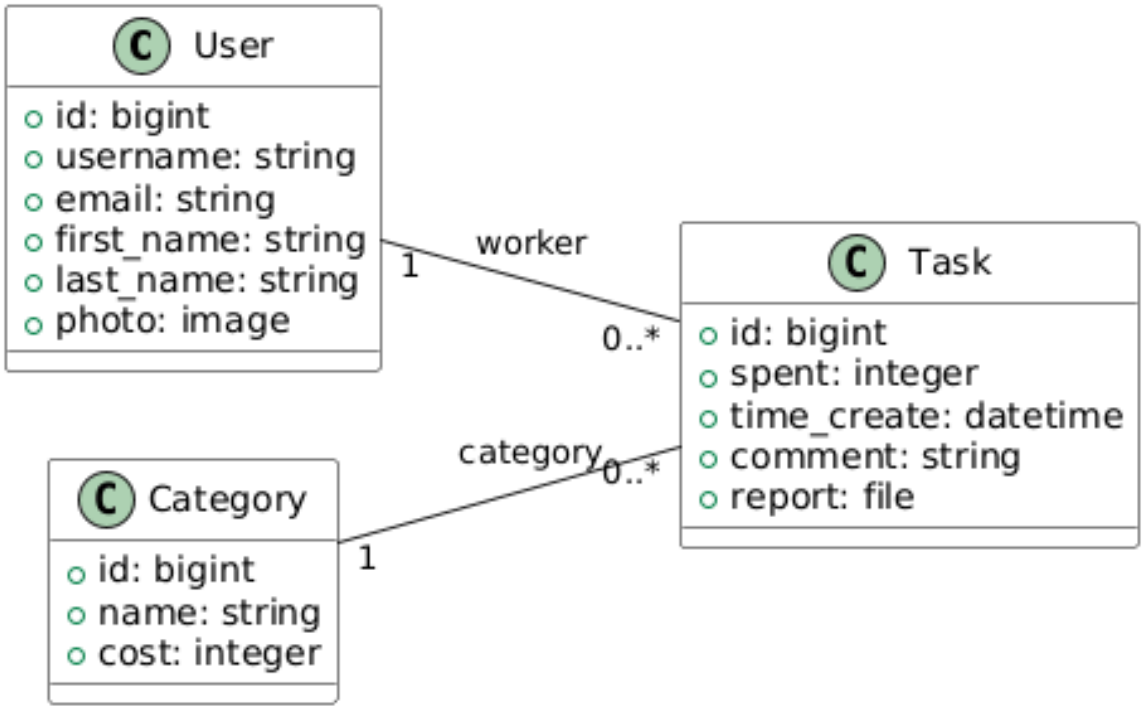

Рисунок 3 – ER-диаграмма веб-сервиса по управлению персоналом

4.4 Компоненты приложения в терминах MVT

В структуре проекта компоненты паттерна MVT представлены следующим образом.

К уровню моделей относятся классы `User`, `Category` и `Task`, расположенные в модуле `Tasks/models.py`. Они определяют структуру хранимых данных и связи между объектами.

К уровню представлений относятся классы `HomePage`, `LoginEmployee`, `LoginSupervisor`, `RegisterUser`, `ProfileUser`, `Statistics` и `TasksList`, размещенные в файле `Tasks/views.py`. Эти представления обеспечивают обработку основных пользовательских сценариев: вход в систему,

регистрацию, работу с личным профилем, просмотр общей статистики и детализации по сотруднику.

К уровню шаблонов относятся HTML-файлы `profile.html`, `statistics.html`, `tasks_list.html`, `login.html` и базовый шаблон `base.html`. Они отвечают за визуальное представление данных, подготовленных на уровне представлений.

Маршрутизация запросов организована через файлы `TaskAndTime/urls.py` и `Tasks/urls.py`. На этом уровне URL-адреса связываются с соответствующими обработчиками и тем самым обеспечивается доступ к функциям веб-сервиса.

Компоненты приложения организованы в соответствии с паттерном MVT: модели описывают данные и связи между ними, представления обрабатывают запросы и готовят данные для ответа, а шаблоны формируют итоговые HTML-страницы [16].

4.5 Пользовательские сценарии работы системы

Для проектируемого веб-сервиса можно выделить несколько ключевых сценариев взаимодействия пользователя с системой:

- регистрация нового сотрудника;
- вход сотрудника в систему и просмотр собственного профиля;
- добавление записи о выполненной задаче;
- вход руководителя в систему;
- просмотр общей статистики по сотрудникам;
- просмотр перечня задач выбранного сотрудника.

С точки зрения архитектуры особенно важными являются сценарии добавления задачи и просмотра статистики, так как они задействуют основные сущности предметной области и ключевые механизмы серверной части приложения. В первом случае пользователь передает данные формы в представление, после чего запись сохраняется в базе данных. Во втором случае представление выполняет агрегирующие запросы и передает результаты в шаблон для отображения руководителю.

4.6 Выводы по главе

В результате проектирования архитектуры веб-сервиса установлено, что паттерн MVT позволяет корректно и наглядно организовать структуру приложения на Django. Архитектура сервиса строится вокруг моделей предметной области, обработчиков пользовательских запросов и шаблонов отображения, что соответствует требованиям к серверной части интернет-ресурса.

Были выделены ключевые сущности системы, определены их связи и описаны основные компоненты приложения в терминах MVT. Полученная архитектура обеспечивает основу для реализации пользовательских сценариев, связанных с учетом задач сотрудников, фиксацией трудозатрат и предоставлением руководителю сводной статистики.

5 РАЗРАБОТКА СЕРВЕРНОЙ ЧАСТИ ИНТЕРНЕТ-РЕСУРСА

5.1 Реализация моделей данных

Серверная часть веб-сервиса реализована на фреймворке Django в виде отдельного приложения `Tasks`. Основу логики хранения данных составляют модели, размещенные в файле `Tasks/models.py`.

Модель `User` наследует стандартный класс `AbstractUser`, предоставляемый Django. Такой подход позволяет использовать встроенные механизмы регистрации, аутентификации и разграничения доступа, одновременно расширяя структуру пользователя дополнительным полем `photo` для хранения изображения профиля.

Модель `Category` содержит название категории задачи и стоимость выполнения работ соответствующего типа. Наличие этой сущности позволяет отделить описание типа работ от самих записей о задачах и переиспользовать категории в нескольких задачах.

Модель `Task` предназначена для хранения информации о выполненной работе. В ней задаются исполнитель задачи, категория, затраченное время, дата создания, комментарий и прикрепляемый отчетный файл. Для поля `spent` используются валидаторы, ограничивающие допустимое значение интервалом от 1 до 24 часов. Это позволяет отсечь некорректный ввод еще на уровне серверной модели.

Сформированная модель данных отражает требования предметной области и обеспечивает сохранение как первичной информации о сотрудниках, так и сведений об их деятельности.

5.2 Реализация форм и валидации

Для взаимодействия пользователя с серверной частью приложения в проекте реализован набор Django-форм, описанных в файле `Tasks/forms.py`.

Форма `LoginUserForm` используется для аутентификации и содержит поля имени пользователя и пароля. Для регистрации сотрудника используется форма `RegisterUserForm`, основанная на `UserCreationForm`. Она позволяет вводить логин, адрес электронной почты, имя, фамилию, фотографию и пароль. Дополнительно в форме реализована проверка уникальности

адреса электронной почты, что предотвращает регистрацию нескольких пользователей с одинаковым электронным адресом [14].

Форма `AddTaskForm` применяется для добавления новой записи о задаче. Она предоставляет выбор категории, ввод затраченного времени, комментария и прикрепляемого файла отчета. Благодаря использованию `ModelForm` структура формы напрямую согласована со структурой модели `Task`, что сокращает объем шаблонного кода и повышает согласованность серверной логики.

Таким образом, уровень форм обеспечивает первичную валидацию пользовательских данных и упрощает обработку типовых сценариев ввода.

Работа формы аутентификации сотрудника показана на рисунке 4.

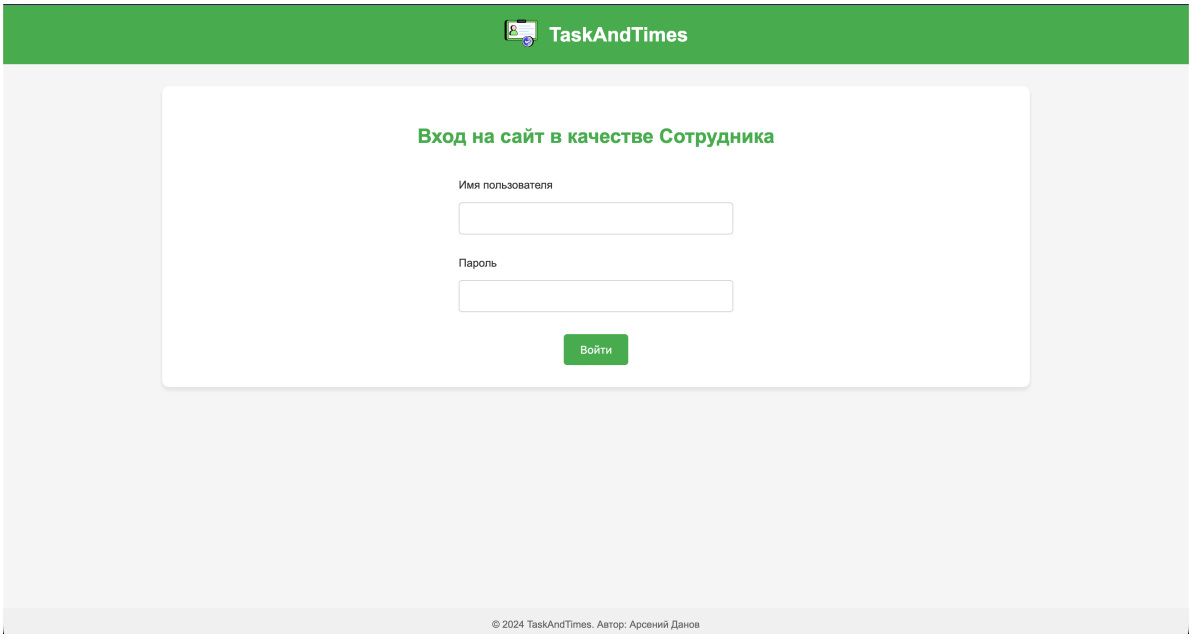

Рисунок 4 – Страница авторизации сотрудника

5.3 Реализация представлений и маршрутизации

Ключевая прикладная логика реализована на уровне представлений, расположенных в файле `Tasks/views.py`. Для маршрутизации используются URL-конфигурации проекта и приложения.

Главная страница реализована классом `HomePage`. Для обработки сценариев входа используются классы `LoginEmployee` и `LoginSupervisor`, наследующие базовую логику `LoginView`. Разделение на два класса входа позволяет задавать разные точки перехода после успешной аутентификации:

сотрудник направляется в личный профиль, а руководитель - к странице статистики.

Регистрация реализована классом `RegisterUser`, основанным на `CreateView`. После успешного создания учетной записи пользователь перенаправляется на страницу входа.

Личный кабинет сотрудника реализован представлением `ProfileUser`, которое одновременно отображает данные текущего пользователя и позволяет добавить новую запись о задаче. В этом представлении формируются агрегированные показатели: количество выполненных задач и суммарное число отработанных часов.

Маршрут `statistics` связан с представлением `Statistics`, реализующим вывод списка сотрудников со сводной информацией. Маршрут `tasks/<int:employee_id>` связан с представлением `TasksList`, которое отображает перечень задач выбранного сотрудника.

Использование class-based views позволило структурировать код серверной части и использовать стандартные механизмы Django для типовых действий чтения, создания и отображения данных [2].

5.4 Реализация аутентификации и разграничения доступа

Для защиты данных и соблюдения требований предметной области в приложении реализованы механизмы аутентификации и разграничения доступа. Вход в систему осуществляется с использованием стандартных средств Django, а доступ к страницам ограничивается с помощью миксинов `LoginRequiredMixin` и `PermissionRequiredMixin` [17].

Страница профиля сотрудника доступна только аутентифицированному пользователю. Это обеспечивает защиту персональных данных и предотвращает доступ посторонних лиц к сведениям о задачах и затраченном времени.

Страницы общей статистики и списка задач конкретного сотрудника защищены проверкой прав доступа. Для их просмотра требуется наличие соответствующего разрешения `Tasks.view_user`. Это позволяет реализовать сценарий, при котором руководитель получает доступ к сводной информации по сотрудникам, а обычный пользователь таких прав не имеет.

Дополнительно в проекте предусмотрены отдельные страницы для обработки ошибок отсутствия доступа и отсутствия ресурса, что повышает завершенность серверной части и делает поведение системы более предсказуемым.

5.5 Реализация статистики и расчетов

Одной из ключевых функций веб-сервиса является предоставление руководителю статистики по сотрудникам. Данная логика реализована в представлении `Statistics` с использованием средств ORM Django.

При формировании списка сотрудников используется агрегирование данных по модели `Task`. Для каждой группы записей, относящихся к конкретному сотруднику, вычисляются:

- общее количество выполненных задач;
- суммарное число отработанных часов;
- расчетная сумма выплат;
- основные сведения о сотруднике.

Расчет суммы выплат осуществляется как произведение затраченного времени на стоимость категории задачи. Для этого используются выражения ORM с объектами `F`, а итоговые значения агрегируются через `Sum`. Такой подход позволяет перенести вычисления на уровень базы данных и уменьшить объем дополнительной обработки в Python-коде [14].

Страница подробной информации о задачах конкретного сотрудника строится на основе представления `TasksList`, где выбираются все задачи по идентификатору сотрудника и передаются в шаблон для отображения.

5.6 Кеширование и запуск приложения

Для ускорения формирования часто открываемых страниц в проекте используется механизм кеширования на основе Redis. В конфигурации маршрутов для страниц профиля, статистики и списка задач применяется декоратор `cache_page`, что позволяет на ограниченное время сохранять сформированный ответ и снижать нагрузку на серверную часть приложения.

Практический запуск веб-сервиса выполняется в контейнеризированной среде с использованием Docker и сопутствующих сервисов. Данное решение упрощает воспроизводимость окружения разработки и развертывания [18].

5.7 Демонстрация работы веб-сервиса

Результаты реализации серверной части подтверждаются работой пользовательских сценариев в веб-интерфейсе приложения. После аутентификации сотруднику доступна страница личного профиля, на которой отображаются основные сведения о пользователе, количество выполненных задач и суммарное количество часов. На этой же странице размещена форма добавления новой задачи.

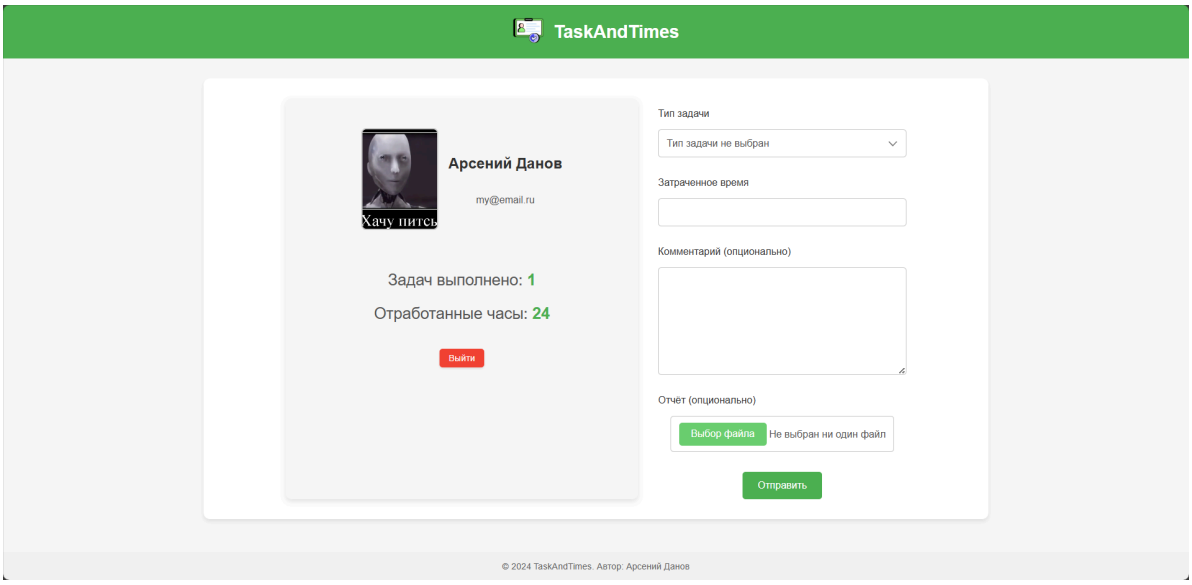

Рисунок 5 – Страница профиля сотрудника

Руководителю доступна отдельная страница статистики, на которой отображается список сотрудников с указанием количества выполненных задач, суммарных трудозатрат и расчетной суммы выплат.

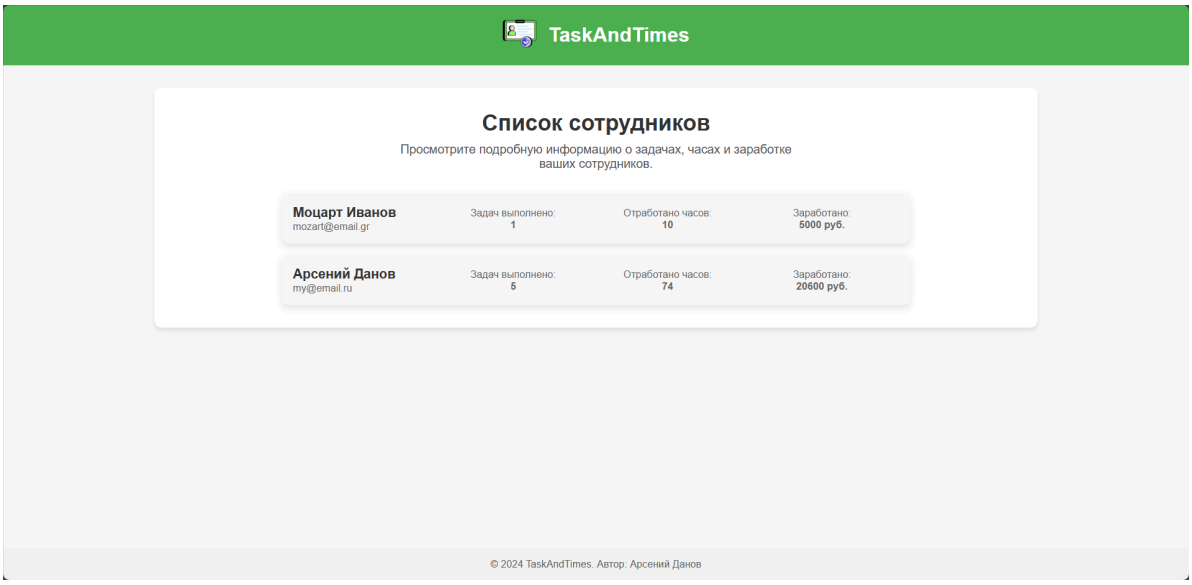

Рисунок 6 – Страница статистики по сотрудникам

При выборе конкретного сотрудника руководитель может перейти к просмотру перечня задач с детализацией по выполненным работам, затраченному времени и прикрепленным отчетам.

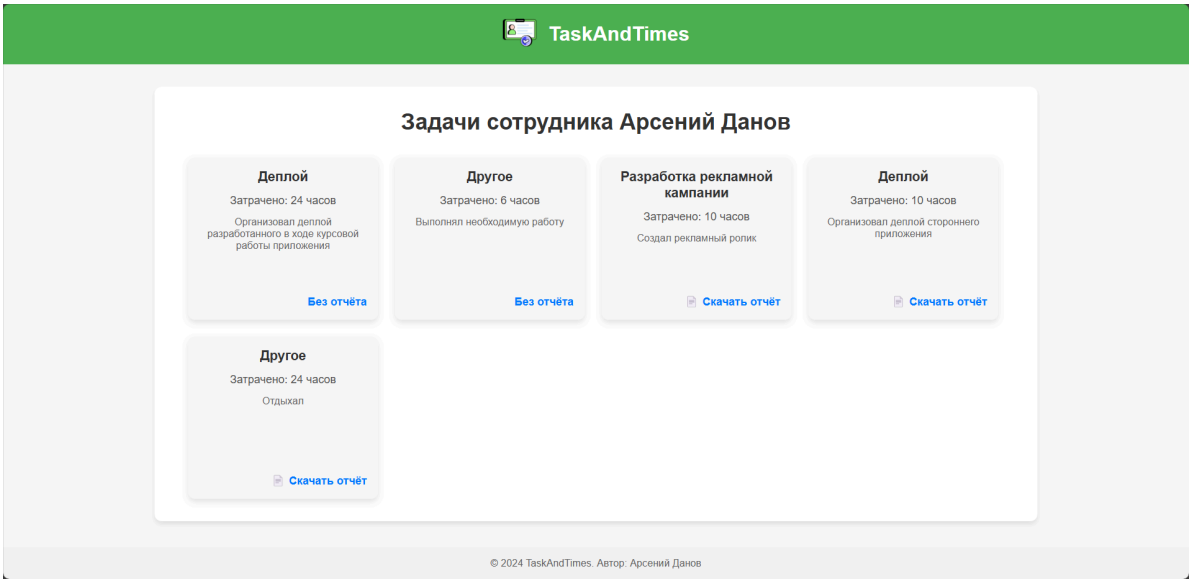

Рисунок 7 – Страница задач конкретного сотрудника

Для сопровождения данных в проекте также используется встроенная административная панель Django, через которую можно управлять моделями пользователей, категорий задач и записей о выполненных работах.

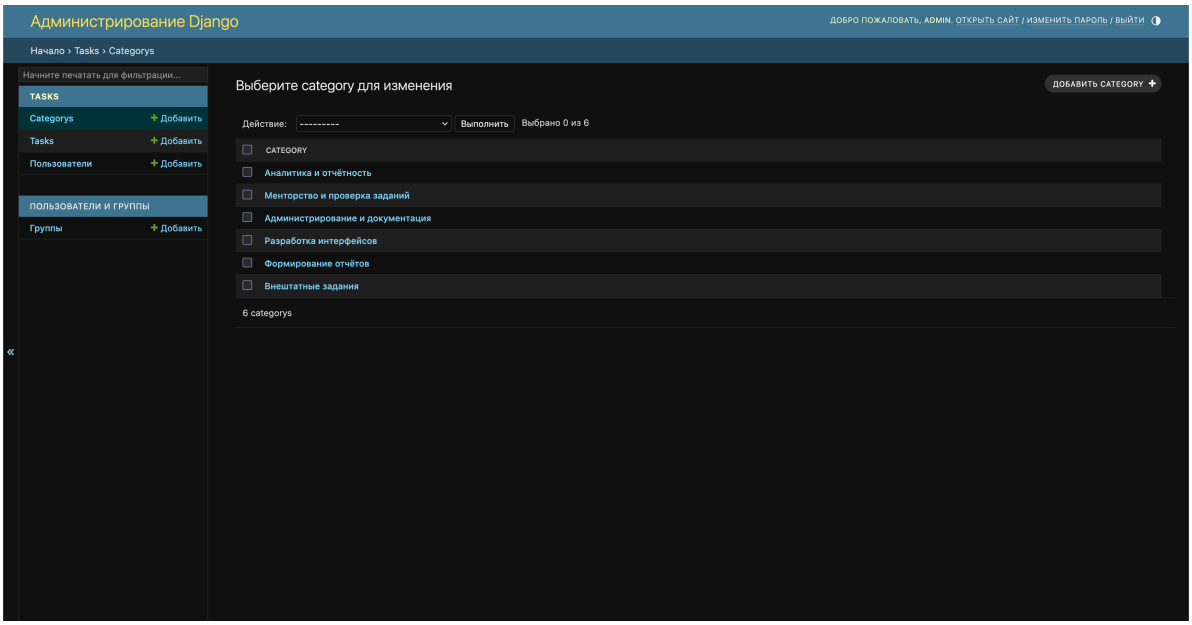

Рисунок 8 – Административная панель Django

5.8 Выводы по главе

В ходе разработки серверной части интернет-ресурса реализованы основные компоненты веб-сервиса по управлению персоналом: модели

данных, формы, представления, маршрутизация, механизмы аутентификации и разграничения доступа. На основе ORM Django реализованы операции хранения и выборки данных, а также агрегатные вычисления, необходимые для формирования статистики по сотрудникам и расчета выплат.

Полученная серверная часть обеспечивает выполнение основных пользовательских сценариев предметной области и подтверждает практическую применимость выбранных технологий и паттерна MVT для разработки веб-сервиса данного класса.

ЗАКЛЮЧЕНИЕ

В рамках курсовой работы по дисциплине «Бэкенд-разработка» был разработан веб-сервис по управлению персоналом, ориентированный на учет сотрудников, выполненных ими задач, затраченного рабочего времени и формирование сводной информации для руководителя. Поставленная цель, заключавшаяся в разработке серверной части веб-сервиса, была достигнута.

В ходе выполнения работы был проведен анализ предметной области, рассмотрены существующие программные решения и сформированы функциональные и нефункциональные требования к разрабатываемому приложению. Это позволило определить состав основных участников системы, ключевые пользовательские сценарии и необходимый набор функций серверной части.

На следующем этапе были обоснованы технологии разработки веб-приложения. В качестве основного серверного фреймворка выбран Django, в качестве системы управления базами данных - PostgreSQL, в качестве средства кеширования - Redis. Для архитектурной организации приложения выбран паттерн MVT, соответствующий используемому фреймворку и задачам разработки серверной части интернет-ресурса.

По результатам проектирования была разработана архитектура веб-сервиса, включающая модели предметной области, обработчики пользовательских запросов, систему маршрутизации и шаблоны отображения данных. В практической части реализованы основные функции серверной части: регистрация и аутентификация пользователей, разграничение доступа, добавление задач, хранение отчетных файлов, формирование личной и сводной статистики, а также расчет суммарных выплат сотрудникам.

Таким образом, результатом курсовой работы является работоспособная серверная часть веб-сервиса по управлению персоналом, реализованная средствами Django и пригодная для дальнейшего развития. Практическая значимость работы заключается в возможности использования разработанного решения как основы для последующего расширения функциональности, внедрения API, улучшения аналитики и развития системы управления персоналом в рамках более крупного интернет-ресурса.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Тузовский А.Ф. Проектирование и разработка web-приложений [Электронный ресурс]: учебное пособие для вузов ISBN978–5–534–515–8. Москва: Юрайт, 2021. 218 с. URL: https://urait.ru/bcode/469982 (дата обращения: 12.05.2026).
2. Меле А. Django 4 в примерах [Электронный ресурс]: руководство ISBN978–5–93700–204–4. Москва: ДМК Пресс, 2023. 800 с. URL: https://e.lanbook.com/book/348113 (дата обращения: 12.05.2026).
3. Антонова И.И., Кашкин Е.В. Разработка web-сервисов с использованием HTML, CSS, PHP и MySQL [Электронный ресурс]: учебно-методическое пособие. Москва: РТУ МИРЭА, 2019. URL: http://library.mirea.ru/secret/15052019/2022.iso (дата обращения: 12.05.2026).
4. Tempo Timesheets [Электронный ресурс]. 2026. URL: https://www.tempo.io/product/timesheets (дата обращения: 12.05.2026).
5. Битрикс24 [Электронный ресурс]. 2026. URL: https://www.bitrix24.ru/ (дата обращения: 12.05.2026).
6. Clockify [Электронный ресурс]. 2026. URL: https://clockify.me/ (дата обращения: 12.05.2026).
7. Flask Documentation [Электронный ресурс]. 2026. URL: https://flask.palletsprojects.com/ (дата обращения: 12.05.2026).
8. FastAPI Documentation [Электронный ресурс]. 2026. URL: https://fastapi.tiangolo.com/ (дата обращения: 12.05.2026).
9. PipeOps. PostgreSQL and MySQL: Which is Better? A 2024 Comparison. 2024.
10. Redis Documentation [Электронный ресурс]. 2026. URL: https://redis.io/docs/latest/ (дата обращения: 12.05.2026).
11. Годзурас Э. Docker Compose для разработчика [Электронный ресурс]: руководство ISBN978–5–93700–203–7. Москва: ДМК Пресс, 2023. 220 с. URL: https://e.lanbook.com/book/348110 (дата обращения: 12.05.2026).
12. Johansson A. HTTP Load Balancing Performance Evaluation of HAProxy, NGINX, Traefik and Envoy with the Round-Robin Algorithm. 2022.
13. Chihana S. Web Server Performance of Apache and Nginx: A Systematic Literature Review. 2018.

14. Django documentation [Электронный ресурс]. 2026. URL: https://docs.djangoproject.com/en/5.1/ (дата обращения: 12.05.2026).
15. Мартин Р. Чистая архитектура. Искусство разработки программного обеспечения. Санкт-Петербург: Питер, 2021. 352 с.
16. Персиваль Г., Грегори Б. Паттерны разработки на Python. TDD, DDD и событийно-ориентированная архитектура. Санкт-Петербург: Питер, 2022. 336 с.
17. Хоффман Э. Безопасность веб-приложений. Санкт-Петербург: Питер, 2021. 336 с.
18. Docker Docs [Электронный ресурс]. 2026. URL: https://docs.docker.com/ (дата обращения: 12.05.2026).